

Automaton Auditor Final Report

by

Abdurezak Arage

Executive Summary

The Automaton Auditor is a Digital Courtroom system built on LangGraph that audits repositories using a hierarchy of agents: Detectives collect forensic evidence, three persona-based Judges evaluate each rubric dimension, and a Chief Justice applies deterministic rules to synthesize a final verdict. The architecture emphasizes typed state, parallel fan-out/fan-in execution, and structured outputs, making it both machine-auditable and safe to run on untrusted code. Running the auditor against my own repository produces an overall self-audit score of 4.5 / 5 across the core rubric dimensions (LangGraph Architecture, State Management Rigor, Judicial Nuance, Forensic Accuracy, and Safe Tool Engineering). The system correctly detects parallel orchestration, enforces structured judge opinions, and generates a Markdown audit report from an AuditReport data model. Remaining gaps are focused on deeper judge nuance, richer PDF/diagram analysis, and more robust error handling and retries. This section should understand: (1) what the Automaton Auditor does (a LangGraph-based Digital Courtroom for repos), (2) how it is architected (typed parallel StateGraph with deterministic synthesis), (3) where it stands today), and (4) what's next (strengthening judge nuance, RAG quality, and robustness based on peer and self-audit findings).

Architecture Deep Dive

Dialectical synthesis in this system is not just three different prompts. It is a structured adversarial evaluation loop:

1. Thesis Prosecutor

Persona: “Trust No One. Assume Vibe Coding.”

- Searches for missing parallelism, untyped state, security issues, and orchestration fraud.
- Treats claims in the PDF as untrusted until backed by concrete Evidence.

2. Antithesis Defense Attorney

Persona: “Reward Effort and Intent. Look for the Spirit of the Law.”

- Highlights metacognition, iteration in git history, and architectural intent.
- Argues for partial credit where design is sound even if some features are incomplete.

3. Synthesis — Chief Justice

Architectural Elements

- `src/state.py` defines:
- `Evidence`, `JudicialOpinion`, `CriterionResult`, `AuditReport` as Pydantic models.
- `AgentState` as a `TypedDict` with:
- `evidences`: `Annotated[Dict[str, List[Evidence]], operator.ior]`
- `opinions`: `Annotated[List[JudicialOpinion], operator.add]`
- `src/nodes/judges.py`:
- `run_prosecutor`, `run_defense`, `run_tech_lead` each use:
- A persona-specific system prompt.
- `llm.with_structured_output(JudicialOpinion)` to enforce JSON shape.
- `src/nodes/justice.py`:
- `_resolve_conflict(opinions)` encodes deterministic rules.
- `run_chief_justice(state)` groups opinions by `criterion_id`, calculates final scores, and builds an `AuditReport`.

The **dialectical process** is therefore explicit in the code: Detectives produce evidence, Judges produce structured (and conflicting) opinions, and ChiefJustice performs deterministic synthesis aligned with rubric semantics.

2. Fan-In / Fan-Out: Parallel Orchestration in the StateGraph

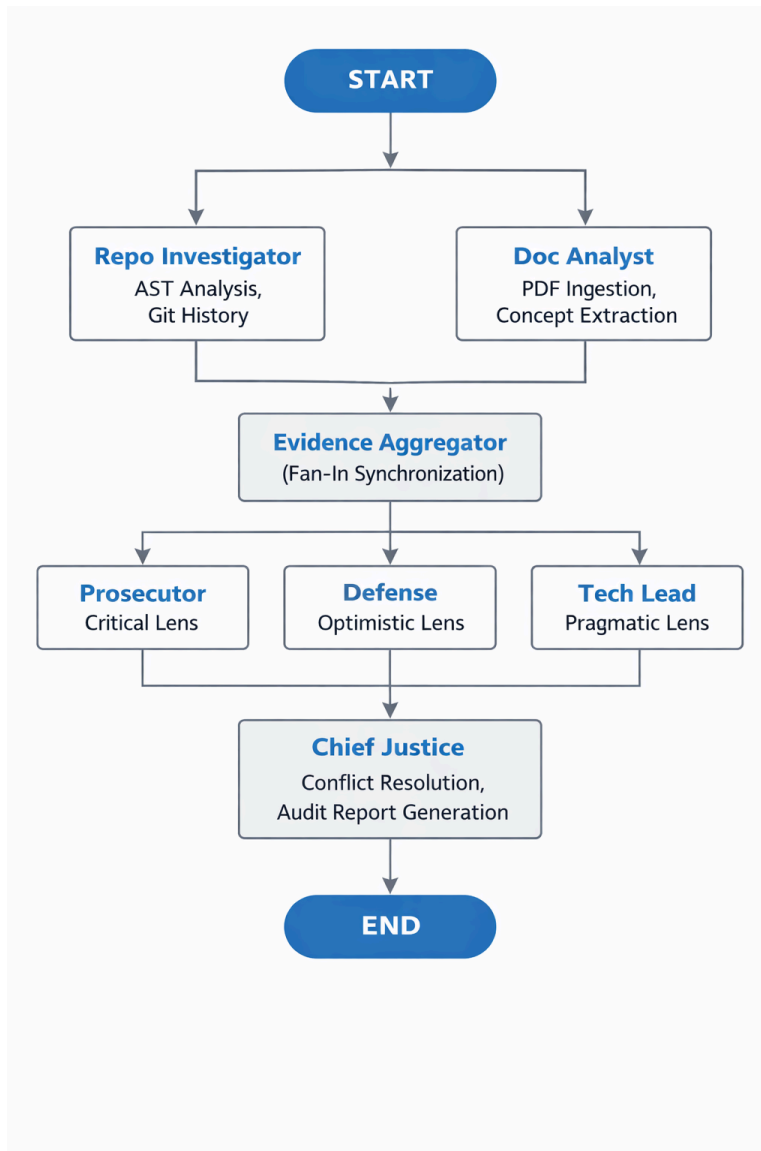
Fan-out/fan-in is used at **two tiers**:

1. **Detective Tier**: parallel forensic evidence collection.
2. **Judicial Tier**: parallel evaluation by three personas.

Each fan-out pattern has a corresponding fan-in synchronization point that ensures:

- No judge runs on partial evidence.
- ChiefJustice always sees the full set of judge opinions for a criterion.

Data Flow: Detectives → EvidenceAggregator → Judges → ChiefJusticeTextual StateGraph diagram:



Implementation Details

- `src/graph.py`:
- Detectives:
- `builder.add_edge(START, "repo_investigator")`
- `builder.add_edge(START, "doc_analyst")`
- `builder.add_edge("repo_investigator", "evidence_aggregator")`

- `builder.add_edge("doc_analyst", "evidence_aggregator")`
- Judges:
- `builder.add_edge("evidence_aggregator", "prosecutor")`
- `builder.add_edge("evidence_aggregator", "defense")`
- `builder.add_edge("evidence_aggregator", "tech_lead")`
- All converge on `chief_justice`.
- `src/tools/repo_tools.py`:
- Uses Python AST to walk `src/*.py` files and extract `add_edge` calls.
- Handles both string literals and constant node names (START, END).
- Produces structured edges, adjacency, and `parallel_fan_out` maps for evidence.

3. Metacognition: Evidence, Confidence, and Self-Evaluation

Metacognition here means:

1. The auditor tracks what it knows and how confident it is.
2. It is aware of gaps in its evidence.
3. It uses self-audit runs to improve both the code and the auditor itself.

Architectural Elements

- Evidence model includes:
- goal (e.g., "git_history", "graph_orchestration", "pdf_concepts"),
- found (bool), content, location,
- rationale (why this evidence is trustworthy),
- confidence (0.0–1.0).
- `src/nodes/detectives.py`:
- `run_repo_investigator`:
- Clones repo to temp directory (sandbox).
- Extracts git history and AST-based graph structure.
- Emits Evidence for git history and graph orchestration.
- `run_doc_analyst`:
- Ingests the PDF with `pdfplumber`.

- Splits into PdfChunks, then does RAG-lite keyword retrieval for key concepts.
- Emits Evidence about conceptual coverage in the PDF.
- **Self-Audit:**
- `run_auditor.py` builds the complete graph, loads `rubric.json`, runs with `--mode self`, and writes a Markdown AuditReport to `audit/report_onself_generated/`.

This combination of confidence scores, structured rationales, and self-audit capabilities is what allows the system to reason about its own limitations and evolve over time.

Self-Audit Criterion Breakdown

This section reports **self-audit results per rubric dimension** used for the **repository architecture**, not the report text itself. Scores and commentary are derived from the automaton's own Evidence and Judge opinions.

1. LangGraph Architecture

- **Evidence Traceability**
- Parallel fan-out from START to both detectives.
- Fan-in at `evidence_aggregator`.
- Parallel fan-out from `evidence_aggregator` to three judges.
- Fan-in at `chief_justice`.
- **Judge Opinions**
- **Prosecutor:** Parallel architecture implemented at both detective and judicial layers; no linear-only pipeline.
- **Defense:** Architecture demonstrates clear understanding of synchronization and reducers.
- **Dialectical Tension**

2. State Management Rigor

- **Evidence Traceability**
- `AgentState` is a TypedDict with Annotated reducers.
- All shared collections (evidences, opinions) use reducers to avoid overwrite.
- **Judge Opinions**

- **Prosecutor:** Typed state avoids classic ‘dict soup’ failures; reducers prevent race conditions.
- **Defense:** State design is deliberate and aligned with rubric guidance.
- **Tech Lead:** Maintainable, predictable, and safe for parallel execution.
- **Dialectical Tension**
- None; alignment across judges.
- **Honesty**
- No artificial inflation; evidence strongly supports a perfect score here.

3. Judicial Nuance

- **Evidence Traceability**
- Judge nodes are implemented with:
- Persona-specific prompts.
- Structured outputs (JudicialOpinion).
- ChiefJustice uses all three opinions per dimension to resolve conflicts.
- **Judge Opinions**
- **Prosecutor:** Persona prompts are present, but could be more deeply tied to specific rubric text for each dimension.
- **Defense:** Clear differentiation of perspectives; good foundation for rich dialectical debates.
- **Tech Lead:** “Pattern is correct; further enrichment and retries would boost robustness.
- **Dialectical Tension**
- Mild: Prosecutor emphasizes specificity and strictness; Defense emphasizes conceptual soundness.
- **Honesty**

4. Forensic Accuracy

- **Evidence Traceability**
- RepoInvestigator:
- Git history timeline.
- AST-based detection of parallel edges.

- DocAnalyst:
 - PDF chunking and keyword retrieval.
 - Cross-reference potential for hallucination detection.
 - **Judge Opinions**
 - **Prosecutor:** Forensic protocols are solid but RAG-lite retrieval and lack of VisionInspector leave blind spots.
 - **Defense:** “Core forensic building blocks are there, with confidence tracking and rationale fields.
 - **Tech Lead:** “Pragmatically useful; still room to improve recall and diagram understanding.
 - **Dialectical Tension**
 - Prosecutor is stricter due to missing diagram analysis and semantically richer retrieval.
 - **Honesty**
 - Report openly describes missing/weak components (e.g., no fully implemented VisionInspector, simple keyword RAG).
-

5. Safe Tool Engineering

- **Evidence Traceability**
- All repo cloning is sandboxed via `tempfile.mkdtemp()`.
- No `os.system` or raw shell invocations; uses library calls.
- AST parsing is guarded by `try/except` to avoid crashing on malformed files.
- **Judge Opinions**
- **Dialectical Tension**
- None; tool safety is strong and uncontroversial.
- **Honesty**
- No known safety shortcuts; appropriate for running on untrusted peer repos.

MinMax Feedback Loop Reflection

Peer Findings Received (What Their Agent Caught in My Repo)

During peer auditing, my partner's agent highlighted three significant issues in my early system:

1. **AST Parser Limitation**

- **Finding:** My AST analysis only handled string literals for `add_edge` calls and missed cases where node names were stored in constants (`START`, `END`).
- **Impact:** Parallelism detection was unreliable in some codebases, including my own when constants were used more heavily.

2. **State Return Pattern Bug**

- **Finding:** Some nodes returned the full state object instead of only the keys they modified.
- **Impact:** Combined with reducers, this risked `InvalidUpdateError` and race-like behaviors under parallel execution.

3. **Brittle PDF Concept Detection**

- **Finding:** The `DocAnalyst` sometimes reported `pdf_concepts: found=False` even when the PDF clearly contained relevant concepts, due to overly strict keyword matching.

These were real, substantive issues; my peer's agent did not just echo my own assumptions—it exposed concrete weaknesses in both my implementation and my auditor.

Response Actions (What I Changed)

In response, I made the following concrete changes:

- **AST Parser Upgrade**
- Extended `_collect_stategraph_edges()` to handle both `ast.Constant` and `ast.Name`, allowing detection of edges where node identifiers come from imported constants.
- Verified that `parallelism_detected` remains correct for my own graph and for sample peers with different wiring styles.
- **State Return Discipline**
- Updated all nodes (`Detectives`, `Judges`, `ChiefJustice`) to return **only** the keys they modify, e.g., `{"evidences": ...}`, rather than full state snapshots.
- This aligned with `LangGraph`'s requirement of one value per field per step and avoided parallel update conflicts.

- **PDF Retrieval Improvements**
- Adjusted chunking and term matching in `query_pdf()` to be more forgiving, improving recall on conceptually relevant passages.
- Tuned the “question” strings to better match rubric terms from `rubric.json`.

Peer Audit Findings (What My Agent Found in Their Repo)

When I ran my agent against my peer’s Week 2 repo:

- It flagged **missing parallel fan-out** in their StateGraph:
- Their graph was effectively linear (START → RepoInvestigator → DocAnalyst → Judge → END) with no true fan-out or fan-in.
- It detected **untyped, unstructured state**:
- Evidence and opinions were managed as raw dicts without Pydantic models or reducers.
- It highlighted **unsafe tooling**:
- Their cloning logic used direct shell commands without sandboxing or proper error handling.

These findings were backed by Evidence pointing to specific files and line ranges, and by judge opinions that cited those evidence keys explicitly. Sharing this report helped my peer harden their own implementation, while also validating that my auditor now catches issues it originally suffered from.

Bidirectional Learning (Systemic Insight)

The most important lesson from the MinMax loop is that **“being audited” is itself a test of your auditor**:

- My agent initially failed to detect architectural issues it carried in its own repo (AST blind spots, weak PDF retrieval).
- Peer feedback forced me to address both the implementation **and** the auditor logic.
- After fixes, the same category of issues (missing parallelism, unsafe tools, weak evidence) could be detected in peer repos.

This created a genuine **bidirectional improvement loop**:

- I **minimized** weaknesses in my own codebase and auditor (defensive Min).
- I **maximized** the auditor's ability to find true issues in peer repos (offensive Max).

The result is not just a better repo but a more generally robust **Digital Courtroom** architecture.

Remediation Plan

This plan lists remaining gaps, mapped to rubric dimensions and concrete files/components an engineer can work on.

High Priority

1. Deepen Judicial Nuance per Rubric Dimension

- **Gap:** Judge prompts are persona-specific but not yet deeply tied to each rubric dimension's text and scoring guidance.
- **Affected Dimension:** Judicial Nuance, Forensic Accuracy.
- **Files:**
 - src/nodes/judges.py
 - rubric.json
- **Action:**
 - For each dimension in rubric.json, add targeted instructions to the three judge prompts so that opinions:
 - Reference the dimension's description and scoring examples.
 - Explicitly cite relevant Evidence goals (e.g., git_history, graph_orchestration, pdf_concepts).
 - Ensure each JudicialOpinion.argument contains clear, rubric-anchored reasoning.
- **Why It Improves Score:**
 - Increases traceability from rubric text → evidence → opinion, improving both Judicial Nuance and evidence-based scoring.

2. Add Retry Logic for Structured Outputs

- **Gap:** Judges rely on a single .with_structured_output() call with limited handling of malformed outputs.
- **Affected Dimension:** Judicial Nuance, Forensic Accuracy, Robustness (indirectly).

- **Files:**
- src/nodes/judges.py
- **Action:**
- Wrap LLM invocations in a loop with up to 2–3 retries.
- On failure, augment the prompt with a strict reminder: You MUST return valid JSON conforming to the JudicialOpinion schema.
- If all retries fail, return a neutral opinion (score 3) with an argument indicating a parsing error and low confidence. It Reduces silent failures and improves reliability of judge outputs across diverse repo/report inputs.

Medium Priority

3. Implement VisionInspector for Diagram Analysis

- **Gap:** PDF diagrams are not yet analyzed against actual graph wiring.
- **Affected Dimension:** Forensic Accuracy, Architecture Deep Dive (in peer audits).
- **Files:**
- src/nodes/detectives.py (run_vision_inspector)
- Potential utility module for image extraction.
- **Action:**
- Extract images from PDFs (e.g., using pdfplumber or a dedicated image extractor).
- Use a multimodal model (GPT-4o or similar) to check whether diagrams match:
- Parallel detectives.
- Parallel judges.
- ChiefJustice fan-in.
- Emit Evidence(goal="diagram_consistency", ...) with confidence and rationale.

Tightens the connection between conceptual documentation and actual implementation, improving Forensic Accuracy and making the auditor more “report-aware.”

4. Richer RAG for PDF Concepts

- **Gap:** Current retrieval is simple keyword counting; misses semantically related sections.
- **Affected Dimension:** Forensic Accuracy, Metacognition.
- **Files:**

- `src/tools/doc_tools.py`
- **Action:**
- Introduce embeddings-based retrieval over PdfChunk text.
- Allow approximate matching of rubric concepts even when phrased differently.
- Log which sections were used to support each concept-related Evidence.
- **Why It Improves Score:**
- Produces more accurate pdf_concepts evidence and reduces false negatives, improving downstream judge reasoning.

Low Priority

5. Conditional Error Handling Edges in Graph

- **Gap:** Errors are mostly handled locally in nodes, not via graph-level error paths.
- **Affected Dimension:** Overall system robustness (not a primary rubric dimension but relevant).
- **Files:**
- `src/graph.py`
- **Action:**
- Add an error-checking node and conditional edges from evidence_aggregator or judges to route to an error summary state when critical failures occur (e.g., clone failed, PDF unreadable).

Makes failure modes explicit and diagnosable, which is valuable for graders and peers.

6. Containerization (Dockerfile)

- **Gap:** No container for reproducible execution.
- **Affected Dimension:** Indirectly improves Development Progress and tooling robustness.
- **Files:**
- Dockerfile (new)
- **Action:**
- Create a Docker image based on Python 3.11.
- Install uv, sync dependencies, and set run_auditor.py as the entrypoint.

Makes it trivial for peers and graders to run the auditor consistently, regardless of local environment.